

NEXA — Configurações por Cliente e Engine de Regras (v1)

1. Objetivo

Definir como o sistema permite que cada cliente (incorporadora/loteadora) configure suas próprias regras operacionais sem comprometer:

- consistência do núcleo do sistema
- integridade dos dados
- previsibilidade da operação
- escalabilidade do produto

Este documento estabelece a base da **Engine de Regras do NEXA**.

2. Princípio Fundamental

O NEXA deve ser:

padronizado no núcleo e flexível na operação

Isso significa:

- o fluxo base (negociação → proposta → reserva → venda) é imutável
 - as regras que controlam esse fluxo podem ser configuráveis por cliente
-

3. Níveis de Configuração

As configurações devem existir em dois níveis:

3.1 Nível da Conta (Account)

Aplica-se a toda a operação da empresa.

Exemplo:

- regras gerais de reserva
 - permissões ampliadas
 - política de fila
-

3.2 Nível do Empreendimento (Development)

Aplica-se a um empreendimento específico.

Exemplo:

- prazo de reserva
 - exigência de documentos
 - comportamento da unidade
-

4. Estrutura Técnica

4.1 Tabelas principais

Devem existir no banco:

- `account_settings`
 - `development_settings`
-

4.2 Exemplo de estrutura

```
type AccountSettings = {  
  accountId: string  
  allowBrokerCreateNegotiation: boolean  
  allowConsultantCounterProposal: boolean  
  reservationApprovalRequired: boolean  
  enableUnitQueue: boolean  
}
```

```
type DevelopmentSettings = {  
  developmentId: string  
  reservationExpirationDays: number  
  requireFullClientData: boolean  
  requireDocumentsForReservation: boolean  
}
```

5. Categorias de Configuração

5.1 Regras de Negociação

Controlam como a negociação pode ser iniciada e evoluída.

Configurações possíveis:

- permitir criação de negociação por corretor
 - exigir cliente completo antes de avançar
 - permitir múltiplas negociações simultâneas por unidade
 - exigir vínculo obrigatório com corretor
-

5.2 Regras de Proposta

Controlam a criação e evolução de propostas.

Configurações possíveis:

- permitir contraproposta por consultor
 - permitir edição após envio
 - exigir aprovação para avanço de status
 - permitir múltiplas propostas por negociação
-

5.3 Regras de Reserva

Controlam o comportamento da reserva.

Configurações possíveis:

- exigir aprovação de gestor/diretor
 - prazo de validade (em dias ou horas)
 - permitir prorrogação
 - exigir proposta aceita antes de reservar
 - permitir reserva direta sem proposta (casos específicos)
-

5.4 Regras de Fila de Unidade

Controlam a disputa por unidade.

Configurações possíveis:

- fila ativa ou desativada
 - critérios para entrar na fila:
 - dados básicos
 - dados completos
 - proposta criada
 - visibilidade da fila
 - promoção automática ao liberar unidade
 - possibilidade de intervenção manual (diretor)
-

5.5 Regras de Venda

Controlam o fechamento.

Configurações possíveis:

- exigir reserva antes da venda
 - permitir venda direta
 - estados intermediários obrigatórios
 - critérios para conclusão:
 - contrato
 - pagamento
 - validação administrativa
-

5.6 Regras de Unidade

Controlam comportamento do estoque.

Configurações possíveis:

- permitir múltiplas negociações por unidade
 - bloquear unidade em negociação
 - permitir retorno de unidade vendida (administrativo)
 - atualizar status automaticamente por evento
-

5.7 Regras de Permissão Complementar

Refinam o controle de acesso.

Configurações possíveis:

- corretor pode criar negociação

- consultor pode criar contraproposta
 - administrativo pode concluir venda
 - gestor pode alterar parâmetros específicos
-

6. Engine de Regras (Comportamento)

A engine deve atuar como camada intermediária entre:

- ação do usuário
 - regra aplicada
 - execução do sistema
-

6.1 Exemplo prático

Cenário:

Solicitar reserva

Fluxo:

1. Usuário solicita reserva
2. Sistema consulta configurações:
 - exige proposta aceita?
 - exige dados completos?
 - exige documentos?
3. Sistema valida regras
4. Se aprovado:
 - cria solicitação ou reserva

5. Se não aprovado:

- bloqueia ação
- informa motivo

6.2 Exemplo de lógica

```
if (settings.requireFullClientData && !client.isComplete()) {  
  throw new Error("Client data incomplete")  
}
```

```
if (settings.requireProposalAccepted && !proposal.isAccepted()) {  
  throw new Error("Proposal must be accepted")  
}
```

7. Princípios da Engine

7.1 Centralização

Todas as regras devem estar em um único ponto lógico (domain/services)

7.2 Não duplicação

Nenhuma regra deve ser implementada em múltiplas telas

7.3 Transparência

O sistema deve informar claramente por que uma ação foi bloqueada

7.4 Configurabilidade controlada

Nem tudo deve ser configurável — apenas o que não quebra o núcleo

8. Regras que NÃO podem ser configuráveis

Para preservar integridade do sistema:

- fluxo base (negociação → proposta → reserva → venda)
 - vínculo obrigatório entre entidades
 - unicidade de reserva ativa por unidade
 - necessidade de contexto (conta + empreendimento)
 - registro de histórico e eventos
-

9. Interface de Configuração

Deve existir um módulo:

Configurações

Com separação clara:

- Configurações da Conta
 - Configurações do Empreendimento
-

9.1 Requisitos da interface

- linguagem clara e objetiva
 - impacto da configuração visível
 - validação de inconsistências
 - controle de quem pode alterar
-

10. Auditoria de Configuração

Toda alteração deve gerar registro contendo:

- usuário
 - perfil
 - valor anterior
 - novo valor
 - data/hora
-

11. Critério de Validação

Este documento estará corretamente implementado quando:

- diferentes clientes operarem com regras distintas no mesmo sistema
 - nenhuma regra precisar ser alterada via código
 - o comportamento do sistema for previsível
 - regras não estiverem espalhadas pela interface
-

12. Síntese

A Engine de Regras é o que permite que o NEXA:

- seja vendável para múltiplos clientes
- se adapte a diferentes operações
- mantenha consistência estrutural

Sem essa camada:

- o sistema se torna rígido ou caótico

Com essa camada:

- o sistema se torna escalável e controlado

